

AquaGuard — Bangladesh flood early-warning + IoT pond management

What this is: a final-year capstone project with two connected parts — (1) three machine-learning models that forecast flood risk for 30 river-gauge locations across Bangladesh using only free, public satellite/weather/river data, and (2) an ESP32-based IoT pond-management system (sensors + pumps) that this flood forecasting feeds into. This document is the starting point for anyone new to the project — a professor, a judge, a teammate, or future-you after time away — to get it running and understand what's here.

If you're going to present or demo this project, read this file first, then [Current status](#) so you know exactly what's real and what's still in progress — nothing here is oversold.

Project map

Folder / file	What it is
<code>manuals/</code>	PDF versions of this file + the setup guides below, for offline/printed reading
<code>packages/</code>	The 3 flood model APIs (FastAPI services) — see packages/SETUP_MANUAL.md to run all three at once
<code>frontend-glass/</code>	The dashboard (glassmorphism look) — recommended, most up to date version
<code>frontend/</code>	The same dashboard, neumorphic look — kept in sync with <code>frontend-glass/</code> feature-for-feature
<code>hardware/</code>	The ESP32 + sensors + pumps rebuild — wiring guides, per-sensor test sketches, progress log
<code>reports/</code>	One rigorous PDF report per model (methodology, literature comparison, results, limitations)
<code>backend/</code>	The training pipeline that produced the models bundled inside <code>packages/*/models/</code>
<code>DECISIONS.md</code>	Every significant design decision made on this project, why, and what it costs — written for a pre-defense walkthrough
<code>MODEL_BUILD_PLAN.md</code>	The full chronological build log — the detailed "how we got here," session by session
<code>hardware/HARDWARE_LOG.md</code>	Same idea as above, scoped to the physical hardware rebuild
<code>hardware/FIREBASE_SETUP.md</code>	Step-by-step guide for setting up the free cloud database that will link the hardware to the dashboard
<code>PROJECT_FEATURE_IDEAS.md</code>	Whole-capstone feature ideas not yet committed to a build plan

Running the demo (zero prior context assumed)

Step 1 — Start the 3 model services

```
cd packages
python -m venv .venv
```

Activate it (`.venv\Scripts\activate.bat` on Windows Command Prompt, `.venv\Scripts\Activate.ps1` on PowerShell, `source .venv/bin/activate` on Mac/Linux), then:

```
pip install -r requirements.txt
python run_all.py
```

Leave that terminal open. Full details, troubleshooting, and what each service does: [packages/SETUP_MANUAL.md](#).

Step 2 — Open the dashboard

```
cd frontend-glass
python -m http.server 5502
```

Open <http://localhost:5502> in a browser. (Double-clicking `frontend-glass/index.html` directly usually works too — try that first if you'd rather not run a second terminal command; switch to the server method above only if "Use my location" doesn't prompt for permission.)

Step 3 — Walk through it

1. Pick a station from the dropdown (or click "Use my location").
2. Click "> **Run full analysis**" — this calls all 3 model services at once: - **Flood risk classifier** — flood/no-flood risk at 24h/48h/72h ahead - **River discharge forecaster** — predicted river discharge (m³/s) at the same 3 horizons (its own prediction also feeds directly into the classifier's — see `DECISIONS.md §8` / `MODEL_BUILD_PLAN.md`'s "3-model pipeline" entry for how) - **Flood susceptibility** — how flood-prone the *terrain* is by nature, independent of current weather
3. Below all three, a **Pipeline summary** appears — 3 plain-language sentences stating what each model actually found (deliberately *not* a single LOW/HIGH verdict — see `MODEL_BUILD_PLAN.md`'s entry on why that design was chosen, so the reader weighs the evidence themselves).

Current status

Written plainly, not to oversell what's finished:

- [done] **All 3 flood models**: trained, tested, documented, each with its own PDF report in `reports/`.
- [done] **Dashboard**: real, live calls to all 3 models; the 3-model pipeline (cascade + combined summary) is built and working.
- [done] **Hardware sensor readings + pump control on the dashboard**: real Firebase reads/writes (`/sensor/*`, `/pumps/*`) — the website side is fully wired and verified live against the real Firebase project. What's still pending is the physical device: 4 of 7 sensors are individually wired and confirmed working (see `hardware/HARDWARE_LOG.md`), but `hardware/AquaGuard_v2/ AquaGuard_v2.ino` hasn't been flashed to real hardware yet — once it is, the dashboard needs no further changes, it'll just start showing real numbers.
- [done] **pH calibration page** (`ph-calibration.html`) and **Analytics page** (`analytics.html`, day/week/month historical charts from Firebase's `/history`): both fully wired to the real Firebase project, verified live end-to-end with real writes/reads/cleanup. Same as above — waiting on the physical ESP32 flash, not on any more website code.
- [done] **Public hosting (Vercel)**: `frontend-glass/` is live at frontend-glass-lilac.vercel.app (static, zero-config, redeploy with `vercel deploy --prod` from that folder). **Important limitation**: the model buttons call `127.0.0.1:8000/8001/8002` — each *visitor's own machine*, not a shared server — so they only work for whoever has `packages/run_all.py` running locally on the exact device viewing the page (the 3 model services are local-only by design, a deliberate documented choice, see `MODEL_BUILD_PLAN.md`). Anyone else opening the link sees a "couldn't reach service" message on those 3 buttons — that's expected, not broken. For a live demo to someone

else, run the services on your own machine first, then share the link (or your screen).

For a technical deep-dive / defense

- **reports/** — one PDF per model (`flood-risk-classifier/`, `discharge-forecaster/`, `flood-susceptibility/`), each covering literature review, methodology, results vs. published benchmarks, limitations, and reproducibility steps. `reports/legacy/` holds the original combined pre-split report, kept for history.
- **DECISIONS.md** — every consequential decision (data sources, label design, model family, station coverage, etc.), why it was made, what alternatives were considered, and what limitation it leaves — meant to be read start to finish by someone who wasn't in the room.
- **MODEL_BUILD_PLAN.md** — the raw, dated, chronological working log of the whole build, for anyone who wants the full "how we actually got here" including dead ends and negative results.

